

# IoT-Based Pressure Monitoring and Root Cause Analysis for Leakage Reduction

Quality Improvement Project (QIP 2025)

## Abstract

This project developed and deployed an IoT-based real-time pressure monitoring system to support leakage detection and quality improvement in a production environment. A DFRobot Gravity analog pressure sensor (SEN0257) was integrated with an ESP8266 microcontroller and calibrated using experimentally obtained voltage-to-pressure mapping. The system performed 24/7 data logging to MATLAB ThingSpeak for remote visualization and time-series analysis. Pressure trends were used to identify abnormal behavior consistent with leakage events above an operational threshold of approximately 40 psi. The collected evidence supported root cause analysis that identified mold parting-line issues, enabling corrective actions that reduced customer complaints by approximately 30 % after implementation.

## 1 Project Objectives

- Design and implement a reliable pressure monitoring system for continuous operation.
- Collect and log real-time production pressure data with remote access.
- Establish a calibrated voltage-to-pressure conversion method for the sensor output.
- Detect leakage signatures using pressure trend behavior and threshold analysis.

## 2 System Architecture

### 2.1 Hardware and Cloud Components

- **Pressure sensor:** DFRobot Gravity Analog Water Pressure Sensor (SEN0257), 5 V supply, 0.5 V to 4.5 V output
- **Microcontroller:** ESP8266 (Wi-Fi enabled)
- **Local display:** 16×2 I<sup>2</sup>C LCD for on-site monitoring
- **Cloud logging:** MATLAB ThingSpeak (Field 1: Voltage, Field 2: Pressure)

## 2.2 Data Flow Block Diagram

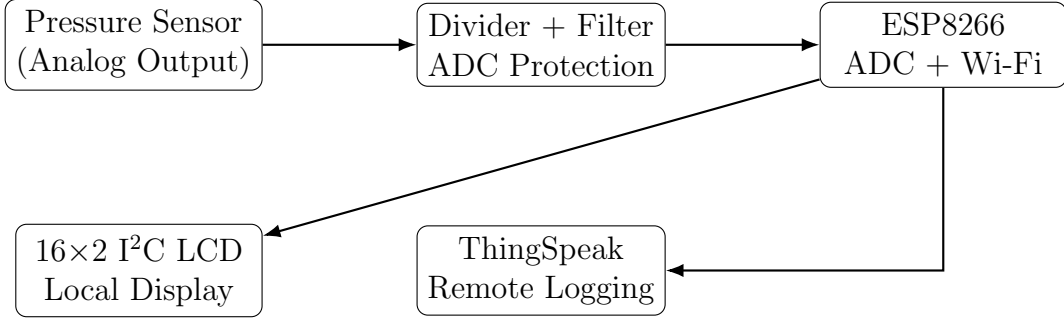


Figure 1: System architecture from sensor acquisition to local display and remote ThingSpeak logging.

## 3 Sensor Integration and Analog Front-End

### 3.1 Sensor Characteristics

The DFRobot SEN0257 sensor provides a linear analog output in the range of approximately 0.5 V to 4.5 V when powered at 5 V. This makes it compatible with typical microcontroller ADCs; however, the ESP8266 ADC input range requires voltage scaling.

### 3.2 Voltage Scaling and ADC Protection

A resistor divider and filtering stage were implemented to safely map the sensor output voltage to the ESP8266 ADC input. The design values used were:

- **R1 (top):** 220 k $\Omega$
- **R2 (bottom):** 47 k $\Omega$
- **Series resistor:** 1 k $\Omega$
- **Filter capacitor:** 100 nF

Divider ratio:

$$V_{ADC} = V_{OUT} \cdot \frac{47}{220 + 47} \approx 0.176 V_{OUT}$$

Thus, 4.5 V corresponds to approximately 0.79 V at the ADC input.

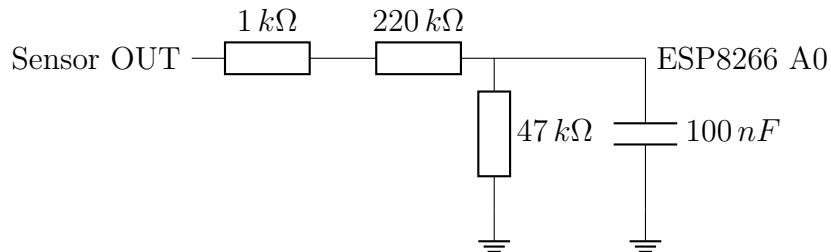


Figure 2: Analog front-end for voltage scaling and ADC stability.

## 4 Calibration and Pressure Conversion

### 4.1 Calibration Data

Calibration was performed by measuring the sensor output voltage and corresponding reference pressure. A piecewise-linear interpolation method was implemented in firmware.

Table 1: Complete calibration points up to 40 psi

| Voltage (V) | Pressure (psi) |
|-------------|----------------|
| 0.406       | 0.0            |
| 0.529       | 8.8            |
| 0.625       | 16.4           |
| 0.635       | 18.0           |
| 0.670       | 20.8           |
| 0.750       | 28.2           |
| 0.812       | 31.0           |
| 0.815       | 32.0           |
| 0.835       | 34.0           |
| 0.859       | 35.8           |
| 0.895       | 38.5           |
| 0.917       | 40.0           |

### 4.2 Calibration Curve

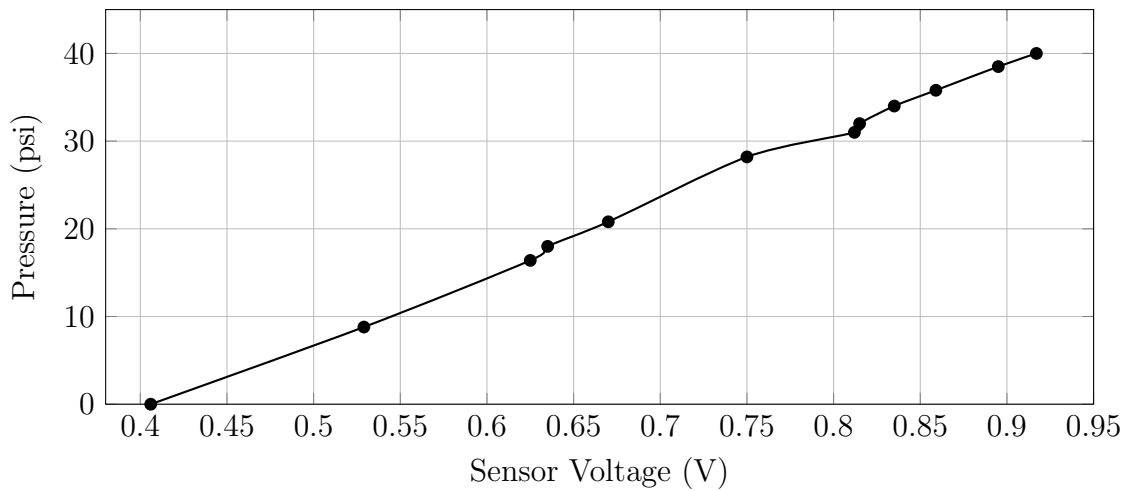


Figure 3: Calibration mapping between sensor voltage and pressure (extended to 40 psi).

## 5 Firmware and Logging Implementation

### 5.1 Firmware Flow

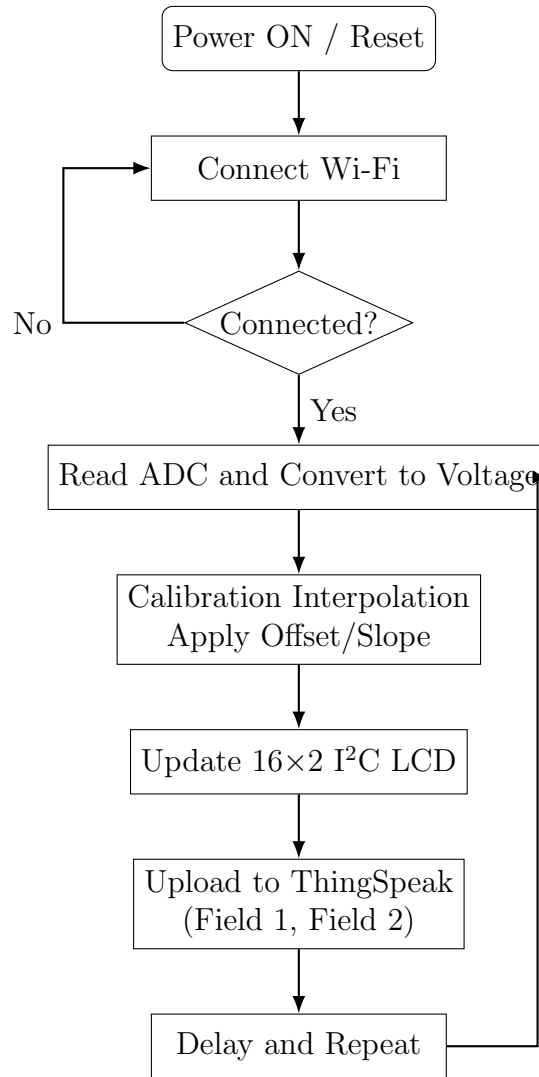


Figure 4: Firmware logic for continuous monitoring, display update, and cloud upload.

### 5.2 Core Calibration Function

Listing 1: Piecewise-linear interpolation used for calibrated pressure output

```
1 float getCalibratedPressure(float v) {
2     for (int i = 0; i < 7; i++) {
3         if (v >= calVoltages[i] && v <= calVoltages[i + 1]) {
4             float slope = (calPressures[i + 1] - calPressures[i]) /
5                           (calVoltages[i + 1] - calVoltages[i]);
6             return (calPressures[i] + slope * (v - calVoltages[i]));
7         }
8     }
9     if (v < calVoltages[0]) return calPressures[0];
10    if (v > calVoltages[7]) return calPressures[7];
}
```

```
11     return 0.0;  
12 }
```

## 6 Time-Series Monitoring and Leakage Detection

### 6.1 Leakage Threshold

During monitoring, leakage events were characterized by abnormal pressure excursions and instability above an operational threshold of approximately 40 psi. This threshold provided a practical rule for identifying abnormal behavior and triggering investigation.

### 6.2 Pressure Trend Snapshot

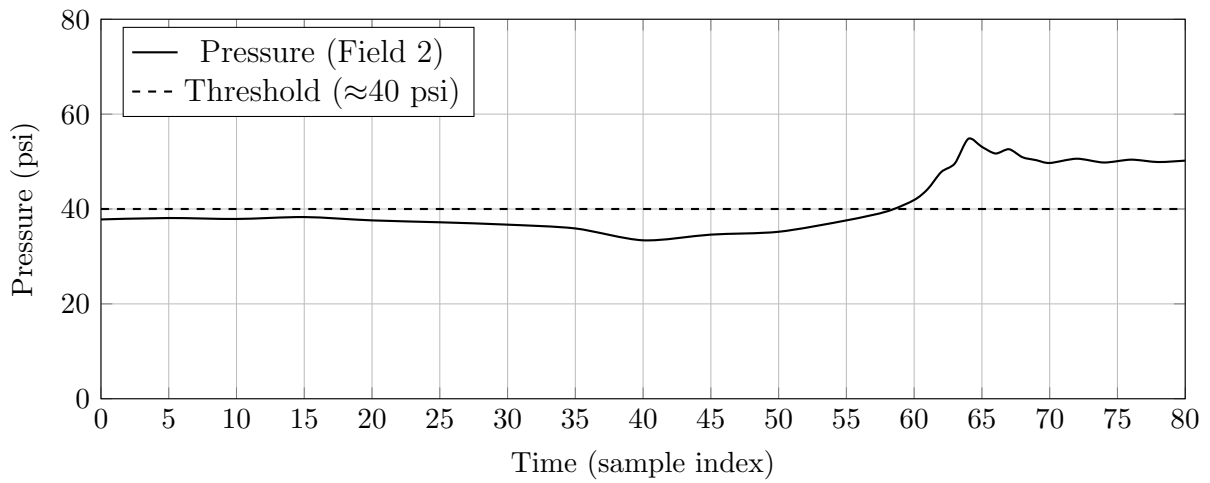


Figure 5: Pressure behavior captured through continuous time-series logging.

### 6.3 Noise-Resistant Trend (Moving Average)

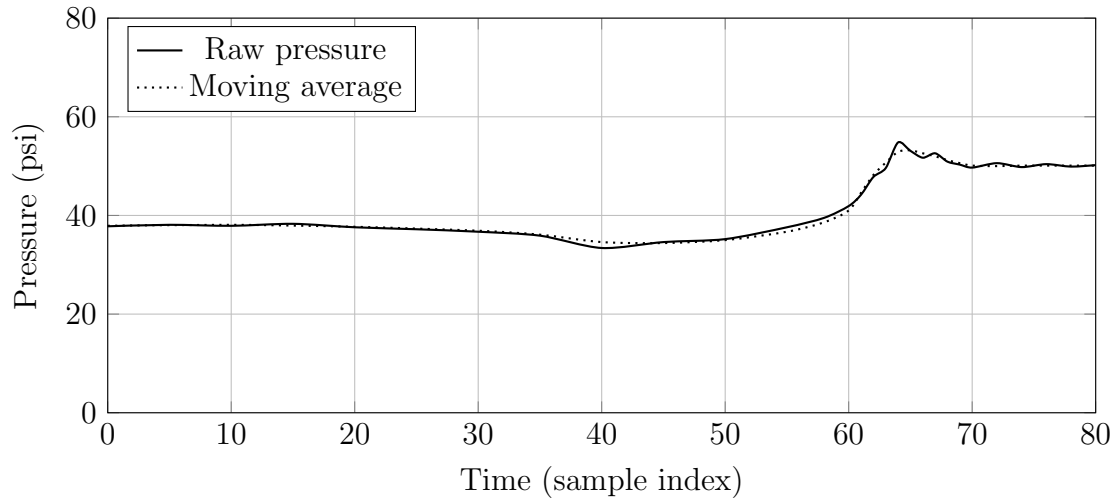


Figure 6: Smoothed trend for stable monitoring and easier anomaly identification.

### 6.4 Leakage Event Count Trend

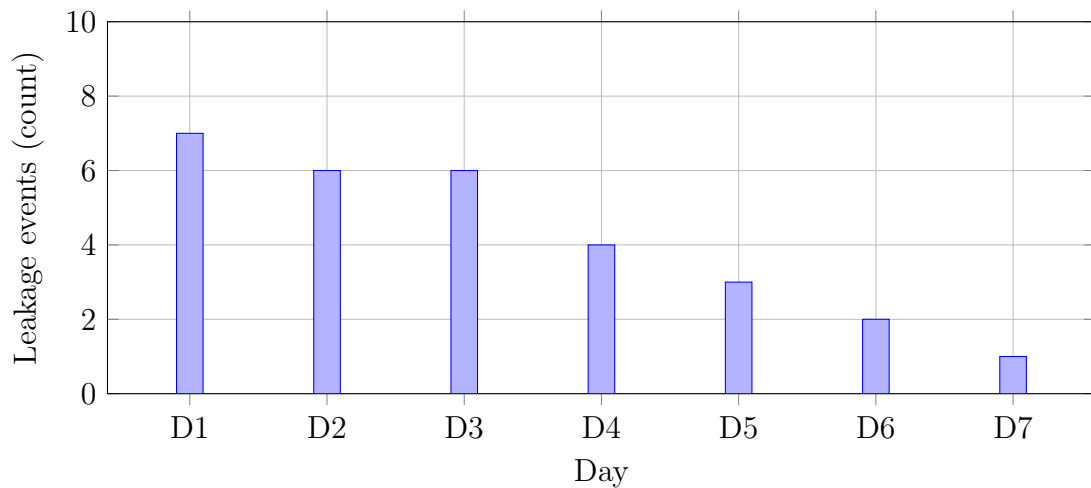


Figure 7: Leakage event reduction trend after monitoring-driven corrective action.

## 7 Root Cause Analysis and Corrective Action

Continuous monitoring enabled correlation of leakage signatures with production conditions. Root cause analysis identified mold parting-line issues as the dominant contributor to leakage-related pressure instability.

Corrective actions included:

- Mold alignment correction and stabilization
- Improved sealing integrity at the parting interface
- Updated inspection and process control steps

Following implementation, customer complaints reduced by approximately 30 %.

## 8 Prototype and Field Implementation Photos



Figure 8: DFRobot Gravity Analog Water Pressure Sensor (SEN0257).

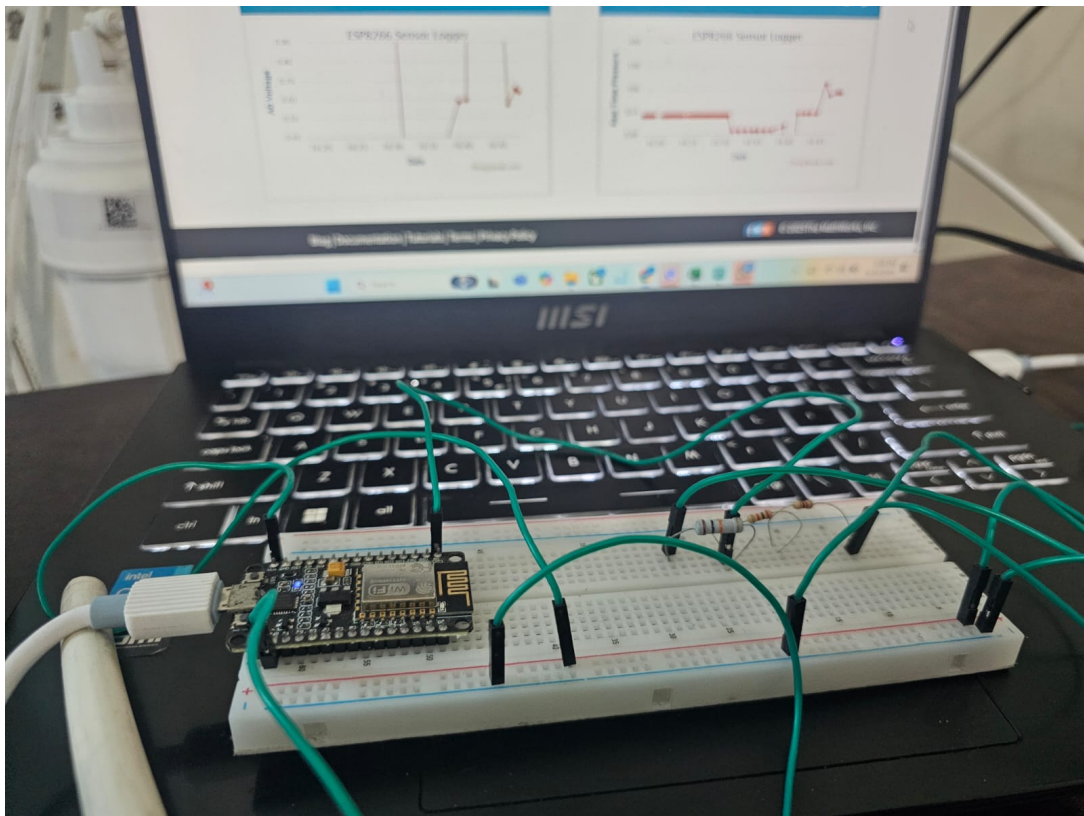


Figure 9: ESP8266-based pressure monitoring prototype with ThingSpeak dashboard.

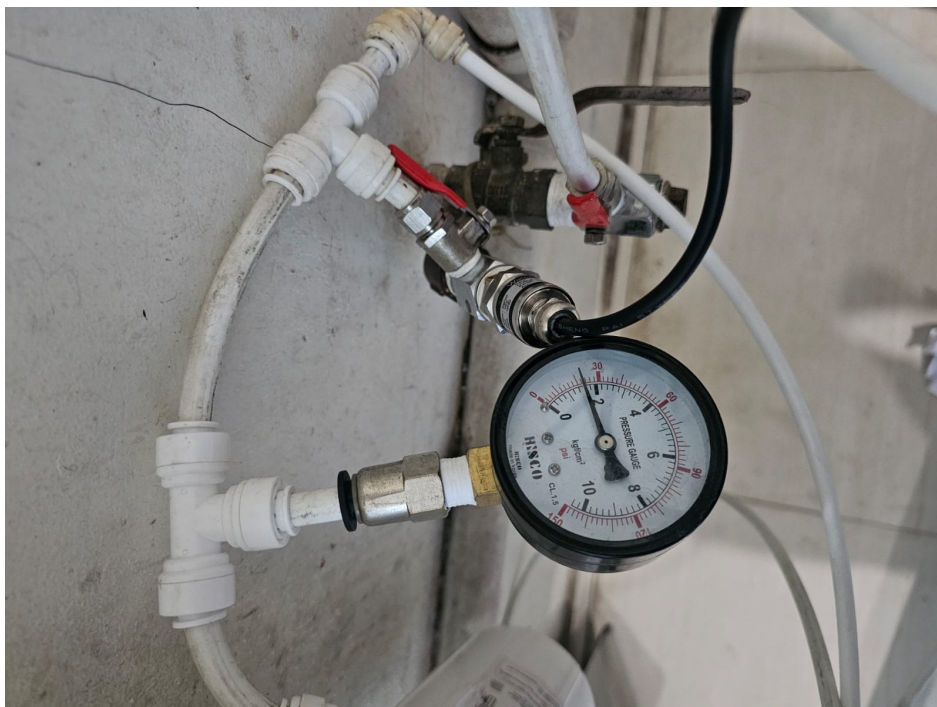


Figure 10: Inline reference gauge used for setup and calibration validation.

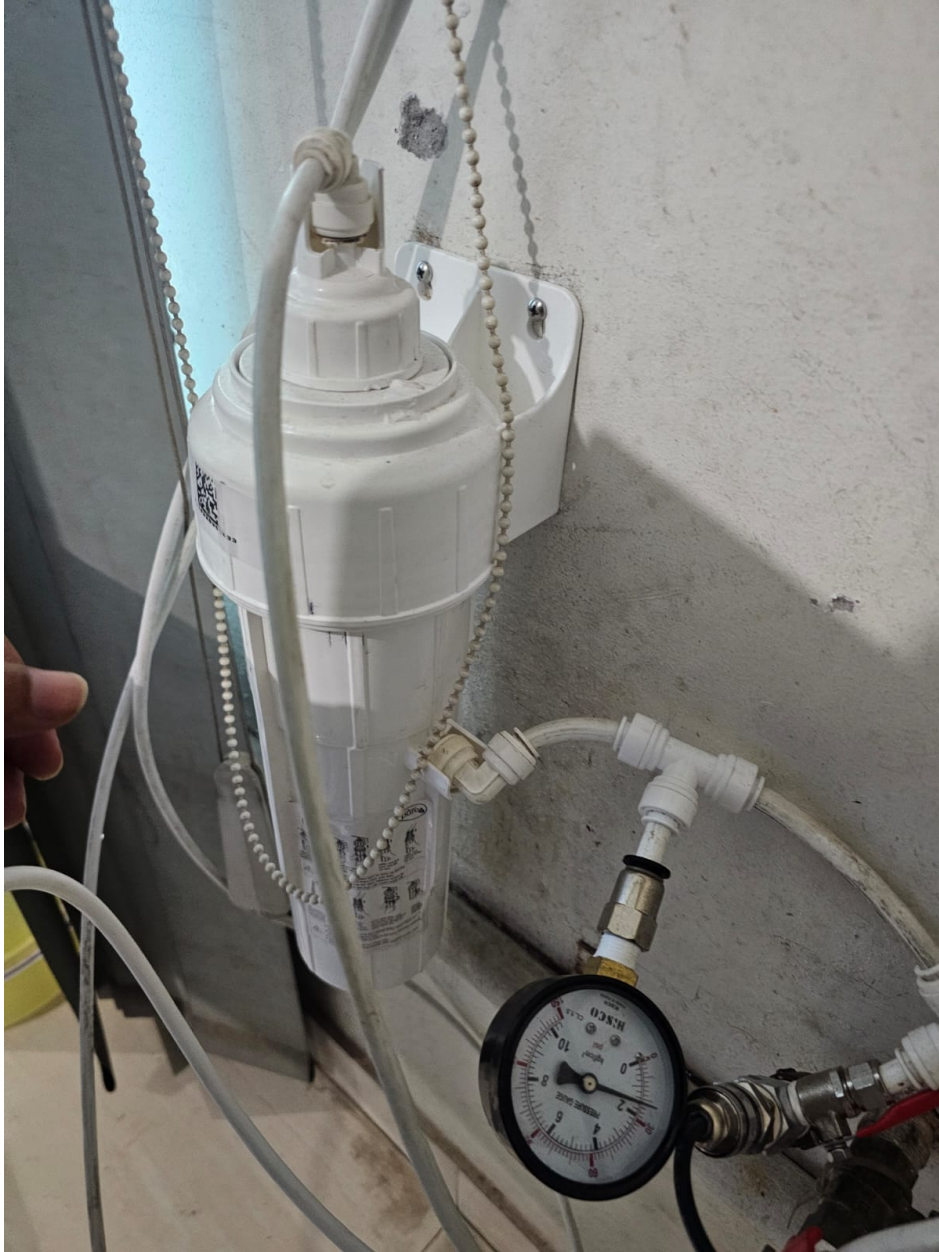


Figure 11: Field installation view showing monitoring point and plumbing configuration.

## 9 Conclusion

An IoT-based pressure monitoring system was successfully designed and deployed using the DFRobot SEN0257 sensor and ESP8266 platform. A calibrated conversion method and continuous ThingSpeak logging provided reliable pressure time-series data with remote access. The collected data enabled structured leakage detection and supported root cause analysis that led to corrective action on mold parting-line issues. After implementation, customer complaints reduced by approximately 30 %, demonstrating the effectiveness of continuous monitoring for industrial quality improvement.